

技术数据

目录

- [1. 仓库结构概览](#)
- [2. 复现步骤](#)
 - [2.1 环境准备](#)
 - [2.2 数据准备](#)
 - [2.3 模型训练](#)
 - [2.4 精度评估](#)
 - [2.5 模拟器评估](#)
 - [2.6 鲁棒性测试](#)
 - [2.7 速度 benchmark](#)
 - [2.8 光计算占比](#)
 - [2.9 远程光子硬件测试](#)
- [3. 关键脚本说明](#)
- [4. 依赖清单](#)
- [5. 编码规范](#)

1. 仓库结构概览

MNIST-train/	
├─ src/	
│ └─ models/	# 模型定义 (base, steq, lsqplus, dsq)
│ └─ quantization/	# 量化核心 (ste, lsqplus, dsq)
│ └─ data/	# 数据加载器
│ └─ training/	# 共享训练循环
│ └─ inference/	# NumPy 推理引擎 + 光子模拟器 + 鲁棒性
│ └─ utils/	# 权重导出/导入
│ └─ scripts/	# 可运行入口脚本
├─ data/	# MNIST 数据集 (.npy)
├─ artifacts/	# 训练产物 (.npy 权重 + 量化参数)
│ └─ ste/	
│ └─ lsqplus/	
│ └─ dsq/	
│ └─ robustness/	
│ └─ reports/	# 本报告生成的结果文件
└─ testing/	# 光子硬件测试 (远程容器运行)

```
├─ docs/
│   └─ ARCHITECTURE/           # 系统架构文档
│       └─ REPORTS/           # 设计/验证/技术 三份报告
├─ requirements.txt           # 依赖清单
└─ CLAUDE.md                  # 项目约定
```

2. 复现步骤

2.1 环境准备

```
# 克隆仓库
cd /path/to/MNIST-train

# 创建虚拟环境
python -m venv venv
source venv/bin/activate # Linux/Mac
# venv\Scripts\activate  # Windows

# 安装依赖
pip install -r requirements.txt
```

2.2 数据准备

MNIST 数据可通过 torchvision 自动下载，或预先生成 `.npy` 文件：

```
# 方式一：通过 torchvision 自动下载（训练脚本自动处理）
# 方式二：手动生成 .npy
python -c "
import torchvision
import numpy as np
from torchvision import datasets, transforms

# 下载
train = datasets.MNIST('./data', train=True, download=True,
transform=transforms.ToTensor())
test = datasets.MNIST('./data', train=False, download=True,
transform=transforms.ToTensor())

# 保存为 .npy
np.save('./data/processed/train_images.npy', train.data.numpy())
np.save('./data/processed/train_labels.npy', train.targets.numpy())
np.save('./data/processed/test_images.npy', test.data.numpy())
```

```
np.save('./data/processed/test_labels.npy', test.targets.numpy())
"
```

2.3 模型训练

全精度基线（3 epochs，约 30 秒）：

```
python src/scripts/train_fp.py
```

STE 量化感知训练（10 epochs，约 2 分钟）：

```
python src/scripts/train_qat_steq.py
```

LSQ+ 量化感知训练（15 epochs，约 3 分钟）：

```
python src/scripts/train_qat_lsplus.py
```

DSQ 量化感知训练（15 epochs，约 3 分钟）：

```
python src/scripts/train_qat_dsq.py
```

训练完成后，权重自动保存至 `artifacts/{ste,lsplus,dsq}/`。

2.4 精度评估

```
# NumPy 推理引擎精度（全部三种方法 + 全精度）
python src/scripts/eval_numpy_accuracy.py

# 输出示例：
# STEQ  NumPy Accuracy: 97.03%
# LSQ+  NumPy Accuracy: 93.18%
# DSQ   NumPy Accuracy: 94.80%
# 结果保存至 artifacts/reports/numpy_accuracy.json
```

2.5 模拟器评估

```
# STE
python src/scripts/run_simulator_steq.py

# LSQ+
python src/scripts/run_simulator_lsplus.py
```

```
# DSQ
python src/scripts/run_simulator_dsq.py

# 模拟器结果保存至 artifacts/reports/simulator_accuracy.json
python src/scripts/save_simulator_results.py
```

2.6 鲁棒性测试

```
python src/scripts/test_robustness_steq.py
python src/scripts/test_robustness_lsplus.py
python src/scripts/test_robustness_dsq.py

# 结果保存至 artifacts/robustness/robustness_test_steq.npy
#                               robustness_test_lsplus.npy
#                               robustness_test_dsq.npy
```

2.7 速度 benchmark

```
python src/scripts/benchmark_inference.py

# 输出：每种方法的 NumPy 推理速度 + 模拟器推理速度
# 结果保存至 artifacts/reports/inference_speed.json
```

2.8 光计算占比

```
python src/scripts/compute_optical_ratio_report.py

# 输出：每层 MAC 数及光计算占比
# 结果保存至 artifacts/reports/optical_ratio.json
```

2.9 光子硬件测试

需在主办方提供的容器中执行：

```
cd /workspace/MNIST-train
source /local/miniconda/etc/profile.d/conda.sh
conda activate moca_llm

python testing/test_steq_photonic.py
python testing/test_lsplus_photonic.py
python testing/test_dsq_photonic.py
```

3. 关键脚本说明

脚本路径	用途	参数/说明
src/scripts/train_fp.py	全精度训练	无参数，自动保存权重
src/scripts/train_qat_steq.py	STE QAT 训练	无参数，10 epochs
src/scripts/train_qat_lsplus.py	LSQ+ QAT 训练	无参数，15 epochs，AdamW+CosineAnnealing
src/scripts/train_qat_ds.py	DSQ QAT 训练	无参数，15 epochs，温度退火
src/scripts/eval_numpy_accuracy.py	NumPy 推理精度	评估全部三种方法，输出 JSON
src/scripts/run_simulator_steq.py	STE 模拟器精度	8x2 tiling 软件回退
src/scripts/run_simulator_lsplus.py	LSQ+ 模拟器精度	含 zero-point 补偿
src/scripts/run_simulator_ds.py	DSQ 模拟器精度	无 zero-point
src/scripts/test_robustness_*.py	鲁棒性扫描	9 级噪声 x 3 次重复
src/scripts/benchmark_inference.py	推理速度测量	warmup + 3 次取均值
src/scripts/compute_optical_ratio_report.py	光计算占比	计算每层 MAC 及总占比
testing/test_steq_photonic.py	光硬件 STE 测试	需 osimulator + entrance
testing/test_lsplus_photonic.py	光硬件 LSQ+ 测试	需 osimulator + entrance

脚本路径	用途	参数/说明
testing/test_dsq_photonic.py	光硬件 DSQ 测试	需 osimulator + entrance

4. 依赖清单

本机依赖

包名	版本	用途
torch	2.9.0	深度学习框架，训练与 QAT
torchvision	0.24.0	MNIST 数据集加载
numpy	2.2.6	NumPy 推理引擎、数据 IO
matplotlib	3.10.7	可视化（对比图）

安装命令：

```
pip install torch==2.9.0 torchvision==0.24.0 numpy==2.2.6 matplotlib==3.10.7
```

光子模拟器内置依赖

包名	版本	说明
numpy	1.22.4	容器内 conda env (moca_llm)
osimulator	1.3.4	光模拟器 API，容器内置
entrance	C 扩展	光子硬件入口，容器内置

注意：osimulator 与 entrance 仅存在于 lightelligence.docker:lt-simulator_v1.4.6 容器中，不可通过 pip 安装。

5. 编码规范

本项目遵循以下编码规范：

1. **文件大小限制**：单文件不超过 200 行（不含注释与 docstring）。
2. **文档语言**：所有 .md 文档采用中英双语（同文件）。
3. **类型注解**：函数参数与返回值均标注类型提示。
4. **模块级 docstring**：每个 .py 文件顶部含概述、References、Consumers。
5. **路径处理**：脚本使用 `__file__` 计算相对路径，确保可从任意工作目录运行。
6. **无占位符**：所有函数必须完整实现，禁止 TODO stub。
7. **脚本薄层**：`src/scripts/*.py` 仅负责导入和调用，核心逻辑在模块中。